

LAB 2: ELECTRIC EYE SAFETY SYSTEM

JOSEPH KRUEGER, CAVAN RILEY, HOLLAND GILMORE, AND BRI
VILLARREAL

A REPORT SUBMITTED AS PART OF THE REQUIREMENTS FOR LAB 2

8 November 2024

ECE:4880
Principles of ECE/CSE Design
Team 15

Contents

1	Design Documentation	1
1.1	Hardware Design	1
1.1.1	Circuit Summary	1
1.1.2	Requirements	2
1.2	Hardware Design: Schematics	2
1.3	Software Design	4
1.3.1	Digital Filtering Algorithm	4
1.3.2	ESP32	5
1.4	Software Design: Schematics	5
2	Design Process and Experimentation	7
2.1	Hardware	7
2.1.1	Receiver Design	7
2.1.2	Provided Parts Integration	7
2.1.3	Hardware Filtering	8
2.1.4	Experimentation	8
2.2	Software	9
2.2.1	Filtering	9
2.2.2	Notifications	10
3	Test Report	12
3.1	Test Strategy	12
3.2	Hardware Tests	12
3.3	Software Tests	13
4	Project Retrospective	14
4.1	Project Management	14
4.2	Gantt Chart	14
4.3	Outcome Summary	15
4.3.1	Hardware	15
4.3.2	Software	15
4.4	Possible Improvements	15
4.4.1	Hardware	16

4.4.2	Software	16
4.5	Member Contributions	16
4.5.1	Cavan	17
4.5.2	Joseph	17
4.5.3	Bri	17
4.5.4	Holland	17
5	Appendix	18
5.1	Figures	18
5.2	Code	18
5.3	References	19

Chapter 1

Design Documentation

1.1 Hardware Design

This section details the design of the Electric Eye Safety System's (EESS) transmitter and receiver modules.

1.1.1 Circuit Summary

The EESS (see Figure [1.1]) consists of a transmitter that emits pulses of infrared (IR) light through two IR light-emitting diodes (LEDs) and an IR photodiode receiver sensitive to the transmitted signal frequency. The system's transmitter, (see Figure [1.2]), was previously developed with a specific transmitting frequency and remains a fixed part of the lab. The process by which the transmitting frequency and other transmitter operating parameters are determined is explained in Chapter 2. In general, the transmitter 555 timer outputs a square wave with a duty cycle and frequency determined by connected resistors and capacitors. The square wave output activates a transistor, causing the emission of IR light through two IR LEDs.

The system's receiver (see Figure [1.3]), the design and filtering capabilities of which are the focus of this lab report, incorporates improvements made to a provided example. The receiver uses a photodiode sensitive to the same wavelengths as the transmitted light to generate a voltage signal, which is then amplified and directed to the analog-to-digital converter (ADC) of an ESP32 microcontroller. Both the received signal's amplification and its buffering just prior to being digitally converted by the ESP 32 are each accomplished by one of the two op amps in the TLV2472 dual op amp package (see Figure [1.4] for a detailed TLV2472 op amp pin out). After the signal buffering, the ESP 32 processes the input voltage signal using a digital filter tuned to the transmission signal frequency, determines if the direct line of sight between the transmitter and receiver is obstructed, and indicates the presence of an obstruction by turning on an LED light and initiating a cellphone notification (see Figure [1.5] for a detailed ESP 32 pin out).

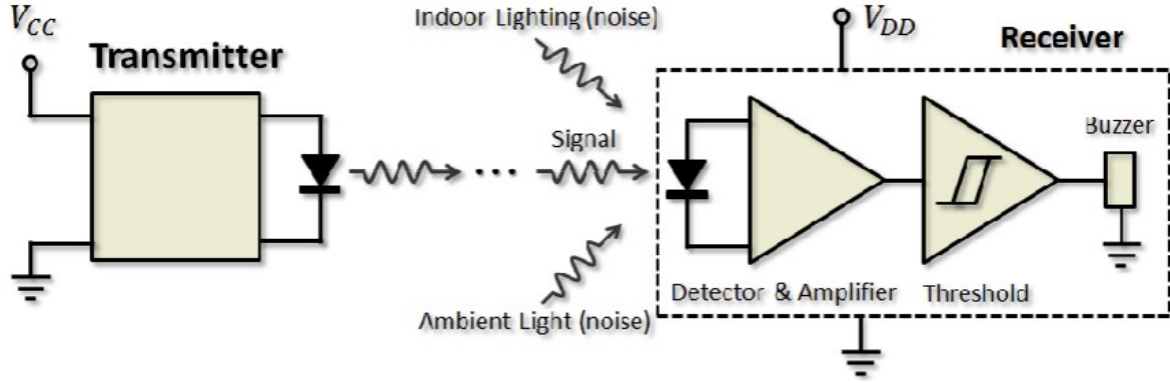


Figure 1.1: Electric Eye Safety System [6]

1.1.2 Requirements

The design flexibility of the EESS lies in its receiver as the transmitter and its operating parameters are predetermined. Design of the receiver is dictated by range, stability, and frequency requirements relative to a lab reference receiver, the attributes of which were measured, as explained in Chapter 2. Generally, system requirements (more specifically receiver requirements) call for the design of an EESS that performs better than the pre-established system with the ability to operate at greater distances between its transmitter and receiver, to be less susceptible to false positives while in the presence of potentially interfering signals, and in a more stable manner.

1.2 Hardware Design: Schematics

Below are schematics for the fixed lab transmitter and the our designed receiver, as well as detailed ESP 32 and TLV2472 op amp connections.

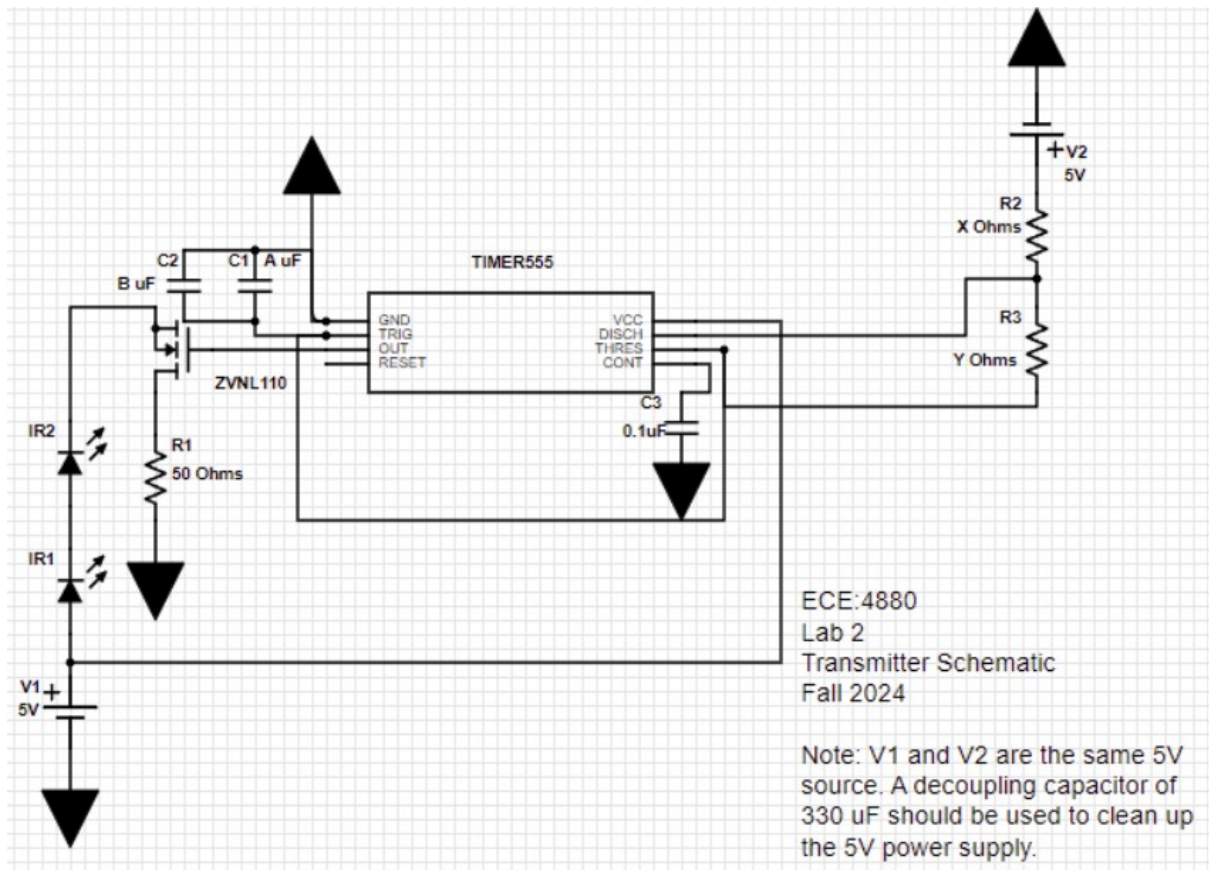


Figure 1.2: Transmitter schematic [3]

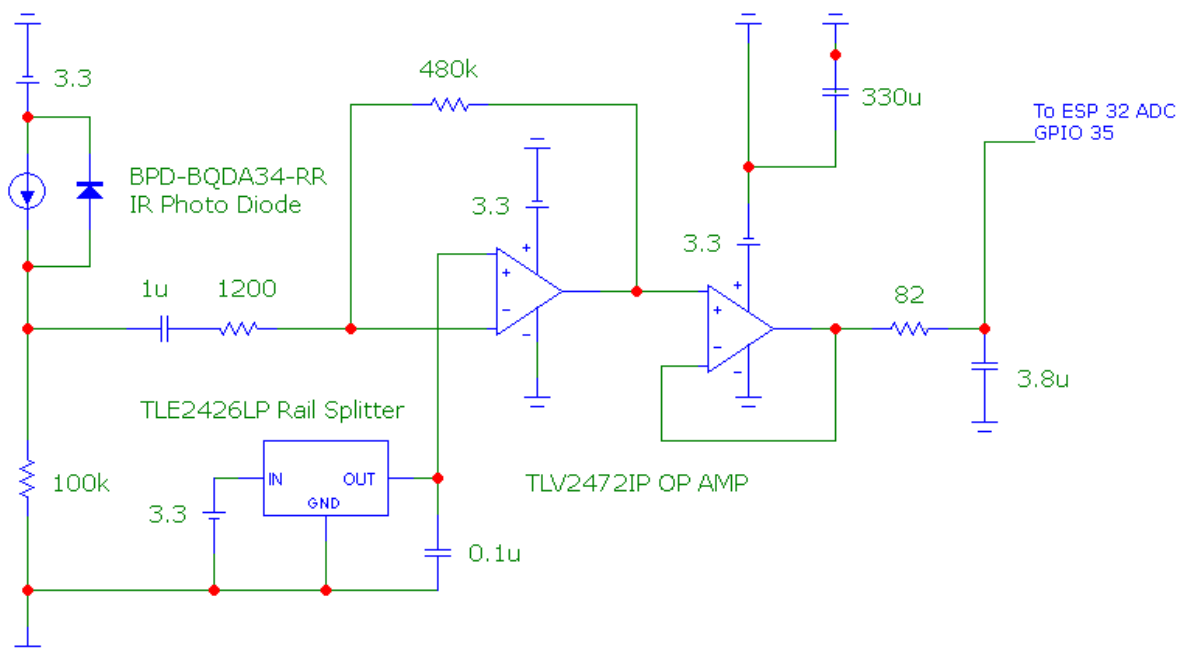


Figure 1.3: Receiver schematic

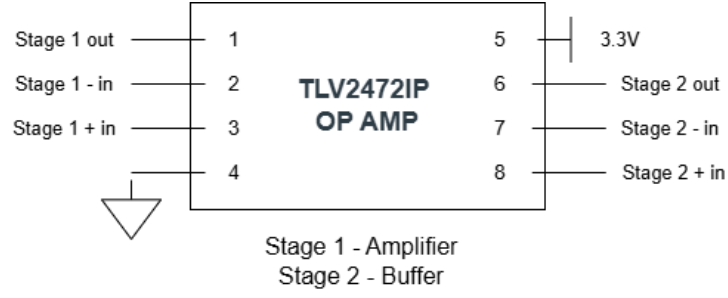


Figure 1.4: Op Amp pinout

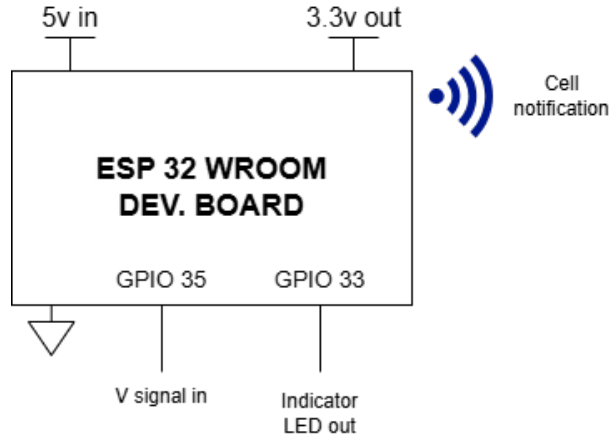


Figure 1.5: ESP 32 pinout

1.3 Software Design

This section details the core software design for the digital filtering algorithm on the ESP32 and the associated notification messaging system.

1.3.1 Digital Filtering Algorithm

The initial software algorithm for the digital filtering system was provided by Professor A. Kruger and Professor R. Mudumbai. The algorithm requires the numerator and denominator coefficients of a custom band-pass elliptic filter [13]. These coefficients are used in the filter difference equation:

$$y[n] = \text{num}[0] \cdot x[n] + \sum_{i=1}^{n-1} (\text{num}[i] \cdot x[n-i] - \text{den}[i] \cdot y[n-i])$$

[1]

We designed this filter using Matlab's Filter Designer package [5.1], and determined the optimal parameters empirically [2.2.1].

1.3.2 ESP32

Filter Implementation

As mentioned previously, the digital filtering code was provided to us. We updated this code to run on the ESP32 instead of the Arduino, which took minimal changes. Moving to the ESP32 allowed for easier implementation of the notification logic, and also allowed for us to implement a filter with a higher sampling frequency due to the faster processor.

The code running on the ESP32 processes input voltages from the internal ADC and stores them into a buffer, and then performing a post-processing hysteresis mechanism to determine a value to filter against. Initially, the provided hysteresis function was a maximum over m samples. We decided to change this to using an average over the last m samples as this should reduce the variance of the filter and thus protecting against noise [2].

We perform filtering by comparing the converted voltage against a threshold value, where any converted voltage beneath the threshold is ignored. The initial threshold provided to us was 0.2, but after optimizing our filter, we found that a 0.15 threshold performed just as well at filtering out noise while allowing us to vastly increase the maximum stable receiving distance.

Notification Implementation

Per the requirements of the lab [6], we were required to implement a notification system for receiver obstruction events. We accomplished this by using googles SMTP server to send an email to an email address hard-coded in the device firmware. We added the alert logic after the hysteresis step of the code, so that if the average received value is below the threshold after 3 hysteresis cycles, we trigger the alert.

1.4 Software Design: Schematics

Below is the high-level software design diagram for the system. This includes the logic for LED status and notification alert triggers.

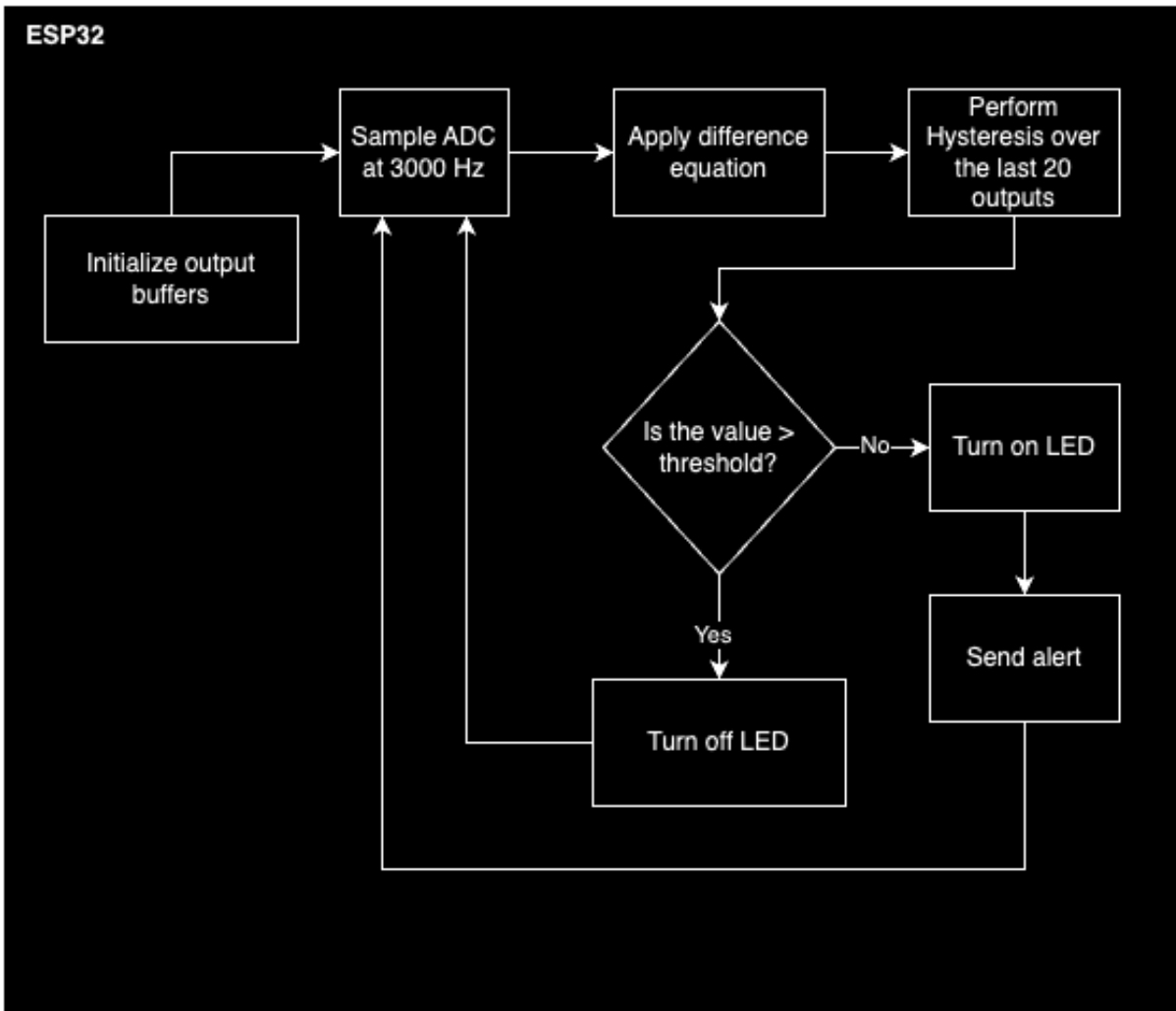


Figure 1.6: Digital Filter Software Implementation

Chapter 2

Design Process and Experimentation

2.1 Hardware

This section outlines the challenges encountered while developing the system hardware and how we overcame them.

2.1.1 Receiver Design

As mentioned previously, the design freedom in this lab was entirely in the system receiver as the established lab transmitter remained fixed. A schematic for both the receiver and the transmitter was provided to be used as a starting point for receiver design and for the construction of a prototype lab transmitter. Additionally, a BPD-BQDA34-RR IR photodiode, two OP165A IR LED's, a TLE2426LP rail splitter, and a TLV2472IP op amp [1] [7] [8] [10] were provided as part of a hardware lab kit which enabled the construction of a receiver based on the provided schematic.

2.1.2 Provided Parts Integration

In keeping with the original intent of the lab, we decided that the filtering done by the receiver to isolate the transmitted signal would be done digitally by a microprocessor and any accompanying hardware would condition signals incoming to the receiver prior to their digital filtering. Initially, a receiver was built true to the provided receiver schematic with an Arduino Uno and a system operating voltage of 5 volts. While this allowed for experimentation and implementation of digital filtering algorithms to begin immediately, the eventual goal was to replace the Arduino Uno with an ESP 32 so that its WiFi capabilities could be exploited to implement cell phone alerts. This replacement would necessitate a receiver system operating voltage of 3.3 volts, a fact which didn't seem to present any problems other than that the provided TLE2426LP rail splitter datasheet [7] advises against input voltages of anything less than 4 volts. Upon trial, however, the TLE2426LP output was measured to be 1.65 volts with an input of 3.3 volts and so an alternate idea to supply the 1.65 volt input bias to the op amp amplifier via a simple voltage divider was disregarded. The subsequent implementation of a 3.3 volt system voltage

and ESP 32 microprocessor was straight-forward.

With the ESP 32 and the TLE2426LP employed as microprocessor and 1.65 volt reference respectively, a decision as to how exactly to use the TLV2472IP op amp needed to be made. The provided receiver schematic showed the op amp configured as an inverting amplifier, while an alternative and perhaps more direct method of converting the receiver BPD-BQDA34-RR IR LED current signal into a voltage signal for amplification would be via the TLV2472IP op amp configured as a transimpedance amplifier. Regardless of the direct applicability of a transimpedance amplifier to our receiver case, we decided, that with careful selection of a relatively large resistor in series with the receiving photodiode, a sufficient voltage signal could be created without any instability issues of a transimpedance amplifier and that the ultimate receiver design would follow in line with the provided receiver schematics' inverting op amp. An inverting gain large enough to drive the amplified signal to a 3.3 volt maximum when the receiver is in close proximity to the transmitter was determined empirically to be approximately 400.

Given that the provided TLV2472IP is a dual op amp package, in addition to the op amp used for an inverting gain of 400 there is one more available for use. Although no experiments were conducted to test the stability or output signal differences between a two-stage amplifier circuit and a single-stage amplifier with a buffer, we decided to use the free op amp as a buffer for the received signal just before entering the ESP 32 ADC.

2.1.3 Hardware Filtering

To perform a crude filtering of the received signal prior to the finer filtering accomplished by the ESP 32 ADC, both a single-stage low pass and a single-stage high pass filter were included in the receiver circuit. At the front end of the receiver a high pass filter couples the received pulsed signal for amplification and has a relatively low cut-off frequency so as not to attenuate the desired received signal frequency of 430 Hz. Just before entering the ESP 32 and after being buffered the signal is conditioned by a low pass filter with a cut-off frequency relatively close to that 430 Hz at about 500 Hz. Additionally, decoupling capacitors were added to the power rails and the output of the rail splitter to remove unwanted high frequency variations.

2.1.4 Experimentation

After building a prototype of the fixed lab transmitter and our designed receiver, there had to be software development to observe how our receiver was working. Using the initial code that was provided, we observed the receiver's functionality by incorporating an LED that would indicate when the photodiode on the receiver could not read a signal.

To assess the efficiency of the lab reference receiver, we used the the fixed lab transmitter operating at approximately 430 Hz. We measured the maximum distance possible between both parts at which the photodiode could still detect a signal, i.e. the maximum distance at which the indicator LED is off. We measured this distance using a tape measurer, recording approximately 21 inches. We aimed to surpass that value by incorporating an enhanced digital

filter and some added hardware filtering.

Using the fixed lab transmitter with our designed receiver, we measured the maximum detection distance to be approximately 39 inches, which is a notable improvement. This confirmed that our designed receiver was functioning more efficiently than the lab reference receiver at the intended frequency of 430 Hz.

While our receiver demonstrated more efficiency at 430 Hz, we also had to verify its ability to filter out noise and frequencies outside our bandpass range, ideally better than the reference receiver did. First, to evaluate the performance of the reference receiver's filter, we used a function generator connected to the fixed lab transmitter, routed to the gate of the ZVNL110 transistor to vary the frequency of the signal.

We conducted tests across a range of frequencies from 100 to 650 Hz, with increments of 50 Hz. At each increment, we recorded the maximum distance at which the signal could still be detected. Analyzing the recorded values, we observed some fluctuations between 100-250 Hz. From 250-650 Hz, the filter's performance became more distinct, with the maximum distance increasing from 250-430 Hz and decreasing from 430-650 Hz. These results confirmed that the reference receiver filter was more efficient at the higher end of the tested range. This allowed us to set an effective goal of demonstrating a filter on our designed receiver that worked from at least 250 to 650 Hz.

Subsequently, with our most efficient filter applied to our system, we repeated the tests using a function generator by varying the frequency and recording the maximum distance that a signal was detected. When comparing these results to those obtained testing the reference receiver, we observed a similar trend. However, we were able to produce lower values at each increment around 430 Hz. In summary, our peak value at 430 Hz outperforms that of the reference receiver and our filter provides more effective noise reduction, therefore meeting the requirements set for our receiver.

2.2 Software

This section outlines challenges encountered while developing the system software and how we overcame them.

2.2.1 Filtering

After developing the initial filtering logic, we needed to improve the system to outperform the receiver in the lab. To do this, we tried various different filter types, orders, and sampling frequencies to determine the optimal filter empirically.

We varied the parameters for the elliptic filter [13] using Matlab's Filter Designer [5.1]. Using this tool we developed various filter architectures and ran the following tests to determine the optimal parameters:

- Measure maximum stable receiving distance with a 430 Hz transmitted signal [1]

- Measure maximum stable receiving distance with a 400 Hz transmitted signal [2]
- Measure maximum stable receiving distance with a 350 Hz transmitted signal [3]
- Measure maximum stable receiving distance with a 500 Hz transmitted signal [4]

To determine the optimal parameters, we chose the filter design that maximized [1] while minimizing the ratio of [2], [3], and [4] to [1]. After running tests on a variety of designs, we determined the optimal parameters to be:

Fs	Fstop1	Fpass1	Fpass2	Fstop2	Astop1	Apass	Astop2	Threshold
3000	300	420	440	700	40	0.5	40	0.15

Table 2.1: Optimal Filter Parameters

Note that the threshold parameter corresponds to the minimum voltage reading to be treated as non-noise. The receiving distances we observed with these parameters were:

430 Hz	400 Hz	350 Hz	500 Hz
39 in	13 in	3.5 in	5 in

Table 2.2: Distance to Frequencies

We found these results to be satisfactory to the requirements of the lab.

2.2.2 Notifications

The main challenge for implementing notifications in this lab compared to Lab 1 was that for Lab 1, we offloaded the notification implementation to the Python Client, whereas in this lab, the notification logic had to be handled by the ESP32 using C++. This posed a challenge as there are much fewer C++ libraries for handling sending notifications than there are for Python.

Our first attempt was to use the Twilio API, but this proved overly complex during early implementation due to connection issues with the API. These issues likely stemmed from struggling to obtain Twilio's root CA certificate preventing a secure connection to be established. This led us to implement the notification logic using the SMTP (Simple Mail Transfer Protocol) for which there is a google server at 'smtp.google.com' allowing for transmission of emails over port 465. We use an email address we created for this course and an app password [17] as the author for the email we send. In order to successfully transmit messages using SMTP, we needed to use the 'ESP Mail Client' Arduino package [11]

As per the lab instructions, the notification shall have the following format:

“Critical Safety Event at HH:MM XX on Month/Day/Year”

- HH is in the range 00 to 12
- MM in the range 00 to 59
- XX is AM or PM

- Month in the range 01-12
- Day in the range 01-31

Obtaining the correct time on the ESP proved more challenging than anticipated, due to the time required for synchronization with NTP (Network Time Protocol) servers that provide the current local time [14]. To remedy this problem, we implemented a waiting mechanism which waits for the synchronization to complete during setup before executing the code within in the loop. We utilize the reliable 'pool.ntp.org' and 'time.nist.gov' servers to retrieve accurate local time information [16].

When connected to the NTP servers we can retrieve the time using the 'getLocalTime' function from the 'time.h' built in C package [15]. The function provides a struct variable with the information we need (the hour, minute, and current date) and we use a custom function 'get-FormattedTimestamp' to format the time in the manner this lab requires. The ESP Mail Client package provides a class for instantiating a SMTP message. We make sure only 1 message is sent when the sensors communication is broken by keeping track of the number of messages sent which may be reset after the sensor begins receiving signal again.

Chapter 3

Test Report

This chapter presents the test reports for the hardware and software components of the device, verifying that our device fulfills the requirements of the lab.

3.1 Test Strategy

Our testing strategy mirrored that of the previous lab, combining physical tests with automated software testing. Due to the nature of this lab, the majority of the tests were physical tests of hardware components and software implementation.

3.2 Hardware Tests

Table 3.1: Hardware Test Report

Test	Description	Method	Result
Target frequency verification	Verify that the receiver can filter the target frequency at a distance greater or equal to that of the lab receiver. We tested 430 Hz	Manual	Pass
Interfering frequency verification	Verify that the receiver can filter interfering frequencies at a distance less than or equal to that of the lab receiver. We tested 400 Hz, 350 Hz, and 500 Hz	Manual	Pass
Transmitter beam interruption	Verify that the LED turns on with little or no delay while the path of light from the transmitter is obstructed	Manual	Pass

3.3 Software Tests

Table 3.2: Software Test Report

Test	Description	Method	Result
Hysteresis function validity	Verify that the hysteresis function returns correct outputs against different inputs	Automated	Pass
Filter stability	Verify that the chosen filter parameters result in a stable filter	Manual	Pass

Chapter 4

Project Retrospective

This chapter reflects on the outcomes of the project, discussing project management strategies and possible improvements for future projects.

4.1 Project Management

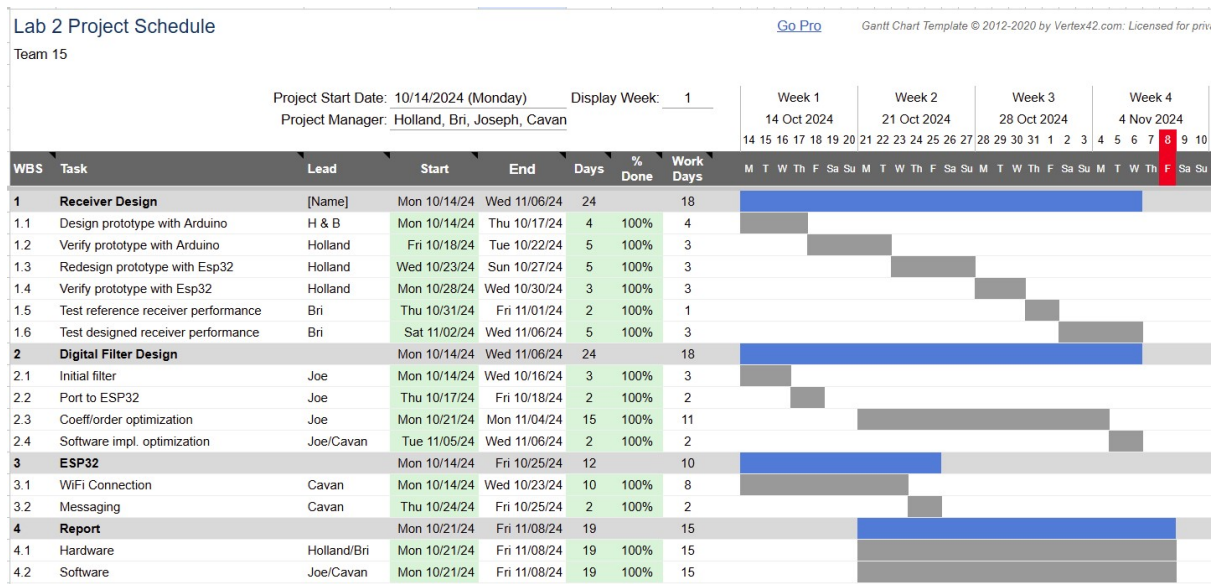
As in lab 1, we implemented a hybrid development model where the hardware development followed a waterfall architecture while the software development followed an agile approach. This resulted in the following model:

1. Develop hardware to requirement standards (waterfall)
2. Once in a stable state, pass off for software development
3. Develop software to requirement standards (agile)
4. Convene to reflect on progress, devise solutions to problems, and test the system
5. Repeat

This model gave the hardware-focused members of the team ample time and ability to develop the hardware up to the standard of requirements, while also giving the software-focused team members the freedom to rapidly iterate on stable hardware. Overall, this model played to the strengths of our team and allowed us to develop fast and efficiently.

4.2 Gantt Chart

As a team, we kept a Gantt chart up-to-date throughout the entire design process to keep track of our objectives list and timelines for completion. The chart is displayed below.



4.3 Outcome Summary

4.3 Outcome Summary

This section outlines the outcome summary after testing the hardware and software components of the system.

4.3.1 Hardware

While we succeeded in nearly doubling the maximum detection distance of the lab receiver, we found that the our receiver was still able to detect a signal when the transmitter was in close proximity and positioned at an exaggerated angle relative to the photodiode. This behavior is undesirable and could potentially render the system unsafe. The hardware design itself was relatively clean and adhered to the size constraints. We also observed the indicator LED staying on for an extended period after the path of light was obstructed and subsequently cleared. While, in comparison, the lab receiver turns the LED off almost immediately after the path is cleared.

4.3.2 Software

While at testing time there were a few issues possibly due to Eduroam connection problems, the software preformed as expected. Once a notification was sent off, the receiver could not resume functioning until the message had finished sending which at times made the LED stay on for prolonged periods of time. Notifications sent to a cell phone were created with the proper time stamp and format and the receiver was not spammed with messages. The filter performed well and almost completely filtered out signals at other frequencies.

4.4 Possible Improvements

This section outlines possible improvements to both the system’s hardware and software components.

4.4.1 Hardware

Firstly, given that test results showed an effective system digital filter, potential improvements to the circuitry could focus on achieving more accurate directionality and a greater functional distance between the system's transmitter and receiver. Some type of mechanical means to direct the pulsed transmission and similarly some means to direct its receipt could increase the sensitivity of the system to the dangers of peripheral obstructions. In this same vein, a more sensitive receiving sensor, perhaps a phototransistor, could also potentially boost system range and decrease the incident angle of permitted transmissions by the receiver. Secondly, given that one of the main problems with the designed receiver was latency associated with ESP 32 Wifi connectivity, future receiver designs could benefit from alternative hardware that could realize cell alerts with less delay. Receiver signal conditioning circuitry could also perhaps be made more efficient, with an op amp of a more targeted functionality and filters with lower time constants, for instance. These hardware decisions would, of course, require significant research and subsequent experimentation.

4.4.2 Software

One possible software improvement is further optimizing the elliptic filter parameters. This could include combinations of changes to the passband width, attenuation, and sampling frequency.

To allow the system to operate with a higher sampling frequency, we could possibly streamline the firmware to run the difference equation as fast as possible. This could include a variety of different software optimizations, such as execution order, print statements, and order of operations in computation. This would allow us to build a higher-order filter that could potentially do a better job at reducing noisy measurements while increasing the maximum receiver distance.

Phone Notifications

Another improvement we could look into would be finding a method that sent notifications quicker to a cell phone. The current implementation is rather slow and disrupts the functioning of the sensors while sending the notifications, however, this may be due to poor connection. A possibility we thought of after the fact was multi-threading the application. A separate thread may send the messages which would keep the time required for sending the message from interfering with the functioning of the sensors. Another possibility we could employ to speed up the sending of the notifications could be to experiment with other packages. Twilio may send messages quicker however due to time constraints we did not think spending the time to configure Twilio would pay off in the end.

4.5 Member Contributions

The following lists each team member's contribution to the lab.

4.5.1 Cavan

Cavan assisted in testing filter options as well as developing the ESP32 communication methodology.

4.5.2 Joseph

Joseph was responsible for assisting in digital filter parameter generation and holistic ESP32 firmware design and development.

4.5.3 Bri

Bri was responsible for assisting in designing the system receiver, constructing the prototype transmitter, as well as experimentation with the hardware.

4.5.4 Holland

Holland was responsible for designing and constructing the system receiver and prototype transmitter circuitry.

Chapter 5

Appendix

This Appendix outlines the figures, pseudocode, and references used in the project.

5.1 Figures

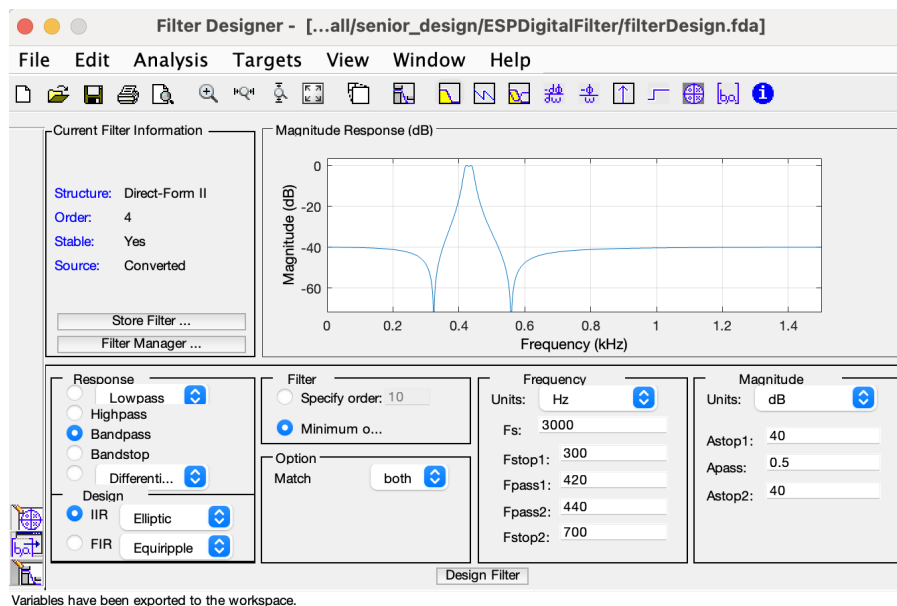


Figure 5.1: Matlab's Filter Designer

5.2 Code

Full codebase: <https://github.com/ECESeniorDesign24-25/ESPDigitalFilter>

Download the codebase [here](#).

Below is the pseudocode for important algorithms and processes implemented in the system software.

Algorithm 1: Filter Difference Equation

Input: Input arrays `num` and `den`; Integers `n` and `m`

Result: Output y_n

```
 $y_n \leftarrow \text{num}[0] \times x[0];$   
for  $i \leftarrow 1$  to  $n - 1$  do  
|  $y_n \leftarrow y_n - \text{den}[i] \times y[i] + \text{num}[i] \times x[i];$   
end
```

Algorithm 2: Hysteresis Function

Input: Input array `arr`; Integer `m`

Result: Output `avg`

```
 $sum \leftarrow 0;$   
for  $i \leftarrow 0$  to  $m$  do  
|  $sum \leftarrow sum + \text{arr}[i];$   
end  
 $avg \leftarrow \frac{sum}{m}$ 
```

5.3 References

1. American Bright Optoelectronics Corporation BPD-BQDA34-RR Pin Photodiode Datasheet. (1998). Available at: <https://www.americanbrightled.com/pdffiles/ir-uv/ir/BPD-BQDA34-RR.pdf>.
2. Diodes Incorporated ZVNL110A: N-Channel Enhancement Mode DMOS FET Datasheet. Available at: <https://www.diodes.com/assets/Datasheets/ZVNL110A.pdf>.
3. ECE 4880 Lab 2 Transmitter Schematic. (2024). Available at: <https://uiowa.instructure.com/courses/237646/files/folder/Lab%20?preview=28353385>.
4. Espressif Systems ESP 32 Series Version 4.7 Datasheet. Available at: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.
5. Espressif Systems ESP-32-WROOM-32 datasheet. (2023). Available at: https://www.espressif.com/sites/default/files/documentation/esp32-wroom_32_datasheet_en.pdf.
6. Laboratory 2, ECE:4880, Principles of ECE Design. (2024). Available at: <https://uiowa.instructure.com/courses/237646/files/folder/Lab%20?preview=28111891>.
7. Texas Instruments TLE2426: The 'Rail Splitter' Precision Virtual Ground. (1998). Available at: https://www.ti.com/lit/ds/symlink/tle2426.pdf?ts=1730672995501&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252FTLE2426.
8. Texas Instruments TLV2472 Dual, 6-V, 2.8-MHz, RRIO operational amplifier Datasheet. (2024). Available at: https://www.ti.com/lit/ds/symlink/tlv2472.pdf?ts=1728921232720&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252FTLV2472.

9. Texas Instruments Xx555 Precision Timers Datasheet. (2024). Available at: https://www.ti.com/lit/ds/symlink/ne555.pdf?ts=1728910534764&ref_url=https%253A%252F%252Feu.mouser.com%252F.
10. TT Electronics Plastic Infrared Emitting Diode OP165 Series, OP166A, OP166B. (2022). Available at: <https://www.ttelectronics.com/TTElectronics/media/ProductFiles/Datasheet/OP165-166.pdf>.
11. ESP Mail Client. Source Code Available at <https://github.com/mobizt/ESP-Mail-Client>.
12. MathWorks. Introduction to Filter Designer. Available at <https://www.mathworks.com/help/signal/ug/introduction-to-filter-designer.html>.
13. Wikipedia. "Elliptic Filter." Available at https://en.wikipedia.org/wiki/Elliptic_filter.
14. Wikipedia. "Network Time Protocol." Available at https://en.wikipedia.org/wiki/Network_Time_Protocol.
15. cplusplus.com. "ctime - C++ Reference." Available at <https://cplusplus.com/reference/ctime/>.
16. NIST. "Official U.S. Time." Available at <https://timegov.nist.gov/>.
17. Google App Password Documentation. Available at <https://support.google.com/accounts/answer/185833?hl=en>.